

SUMMER TRAINING REPORT

on

UNIVERSITY COURSE & EXAM SCHEDULER

TERM - V

Submitted by

PIYUSH KUMAR
Registration Number: 12417684

Under the Guidance of
GAURAV SHARMA
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
LOVELY PROFESSIONAL UNIVERSITY

DECLARATION

I hereby declare that the project work entitled "University Course & Exam Scheduler" submitted to the Department of Computer Science and Engineering, Lovely Professional University, is a record of original work carried out by me under the guidance of Gaurav Sharma. This project, or any part of it, has not been submitted elsewhere for the award of any degree, diploma, or other similar title.

Piyush Kumar
Registration Number: 12417684

CERTIFICATE



CENTRE FOR
PROFESSIONAL
ENHANCEMENT

Certificate No. 491884

NAAC A++

Certificate of Merit

This is to certify that Mr./Ms. **Piyush Kumar** S/D/W/o **Mr. Subodh Kumar**

student of **School of Computing and Artificial Intelligence** Registration No. **12417684**

pursuing **Bachelor of Technology (Computer Science and Engineering)** completed

skill development course named **Data Structures and Algorithm Design using C++**

organized by **Centre for Professional Enhancement** Lovely Professional University

from **14-06-2026** to **26-07-2026** and obtained **A** Grade.

Date of Issue: 13-08-2026
Place of Issue: Phagwara (India)

Prepared by
(Administrative Officer-Records)

Programme Coordinator
Centre for Professional Enhancement

Head of School
School of Computing and Artificial Intelligence

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty guide, Gaurav Sharma, for his valuable guidance, continuous support, and encouragement throughout the course of this project. I am also thankful to the Department of Computer Science and Engineering, Lovely Professional University, for providing the resources and environment necessary to complete this work. Finally, I extend my thanks to my peers and family for their constant support and motivation.

Piyush Kumar

TABLE OF CONTENTS

Declaration i
Acknowledgement ii
1. Introduction to the Project 1
2. Project Content Detail 1
3. Project Detail 1
3.1 Problem Statement 1
3.2 Project Outcomes 2
3.3 Technologies Used 3
4. Source Code / System Snapshots 3
Bibliography 6

1. INTRODUCTION TO THE PROJECT

This report was generated for the University Course & Exam Scheduler, a project built to address real scheduling problems faced by university departments.

The University Course & Exam Scheduler is a C++20 console application that automatically generates conflict-free class timetables and exam schedules from course, faculty, room, and student-batch data. Rather than relying on hardcoded rules, it uses graph algorithms, constraint checking, and greedy/priority-based scheduling to detect and resolve conflicts. The project was originally built for the Smart Education theme of Smart India Hackathon (SIH).

2. PROJECT CONTENT DETAIL

The project models each course as a node in a directed graph, with prerequisite relationships represented as edges between nodes. Kahn's Algorithm is used to perform a topological sort over this graph — an approach built on BFS-style layered traversal using a queue of zero-in-degree nodes — to determine a valid, dependency-respecting ordering of courses before any timetable is generated. The same traversal identifies cycles in the prerequisite graph, so an invalid (circular) course dependency is caught and rejected before scheduling begins, instead of silently producing a broken timetable.

Once a valid course ordering exists, the scheduler layers courses for parallel scheduling and assigns rooms and faculty slots using a priority-queue-based batch prioritization and greedy nearest-fit room allocation. A separate, independent Conflict Engine re-audits the finished timetable for faculty, room, student, prerequisite, and capacity conflicts using hash-map lookups, so the correctness check is not just “trust the generator” — it is verified independently.

3. PROJECT DETAIL

3.1 PROBLEM STATEMENT

University departments must schedule classes and examinations for multiple courses, faculty members, rooms, and student batches every term. Doing this by hand is error-prone: it is easy to double-book a faculty member, assign two classes to the same room at the same time, schedule an exam before its prerequisite course, or ignore room

capacity and lab requirements. The problem addressed by this project is to generate a clash-free class and exam timetable automatically, given course prerequisite dependencies, faculty availability, room constraints, and student-batch enrolments.

3.2 PROJECT OUTCOMES

1. Resolves scheduling conflicts using graph dependency analysis.

2. Produces an optimized, clash-free timetable for a sample dataset.

3. Strengthens understanding of graph traversal techniques (BFS, Kahn's topological sort, DFS-based cycle detection) as applied to real-world scheduling problems.

3.3 TECHNOLOGIES USED

- Language: C++20
- Core data structures: Graphs, Arrays, Queues, Priority Queues, Hash Maps (unordered_map)
- Algorithms: BFS, DFS, Kahn's Topological Sort, Greedy allocation, Backtracking (for exam placement)
- Persistence: JSON-backed storage using the nlohmann/json single-header library
- Build system: CMake (3.16+) and a plain Makefile, targeting g++ 10+ / clang 12+
- Design patterns: Singleton (DataStore), Strategy (pluggable conflict rules), MVC-style separation of entities, engine, and console UI

4. SOURCE CODE / SYSTEM SNAPSHOTS

Note: Only the build configuration (CMakeLists.txt, Makefile) and project documentation (README.md) were provided for this report; the .cpp/.h source files and console screenshots were not supplied, so they are not reproduced here. Insert the relevant source file listings and screenshots of the running application (Admin/Faculty/Student portals, generated timetable, conflict audit, CSV export) in this chapter before submission.

BUILD CONFIGURATION — CMAKELISTS.TXT (EXCERPT)

```
add_executable(scheduler src/main.cpp src/Graph.cpp src/Scheduler.cpp
src/ConflictEngine.cpp src/ExamScheduler.cpp src/DataStore.cpp)
```

The project builds two targets: scheduler (the application) and scheduler_tests (a lightweight, dependency-free assertion-based test suite). The test suite verifies that topological sort keeps prerequisite courses before dependents, that cycle detection correctly rejects an invalid prerequisite graph, that the generated timetable has zero faculty/room/batch conflicts, and that the exam scheduler enforces a minimum one-day gap between exams in the same semester.

SYSTEM / DATA SNAPSHOTS

The figures below are IDE views of the project's data files, generated exports, and build configuration, captured during development. They show the JSON "database" that drives the scheduler and the CSV files it produces, rather than the console application itself running; a live run of scheduler.exe (Admin/Faculty/Student menus) should be captured separately and inserted here before final submission.

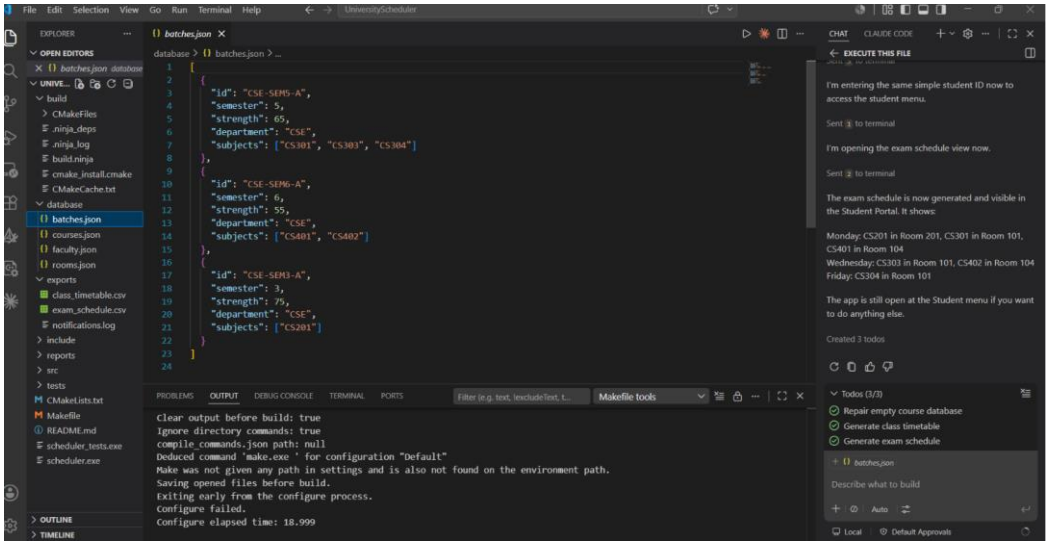


Fig 4.1: batches.json — student batch data (build panel shows a failed CMake configure, unrelated to the JSON content)

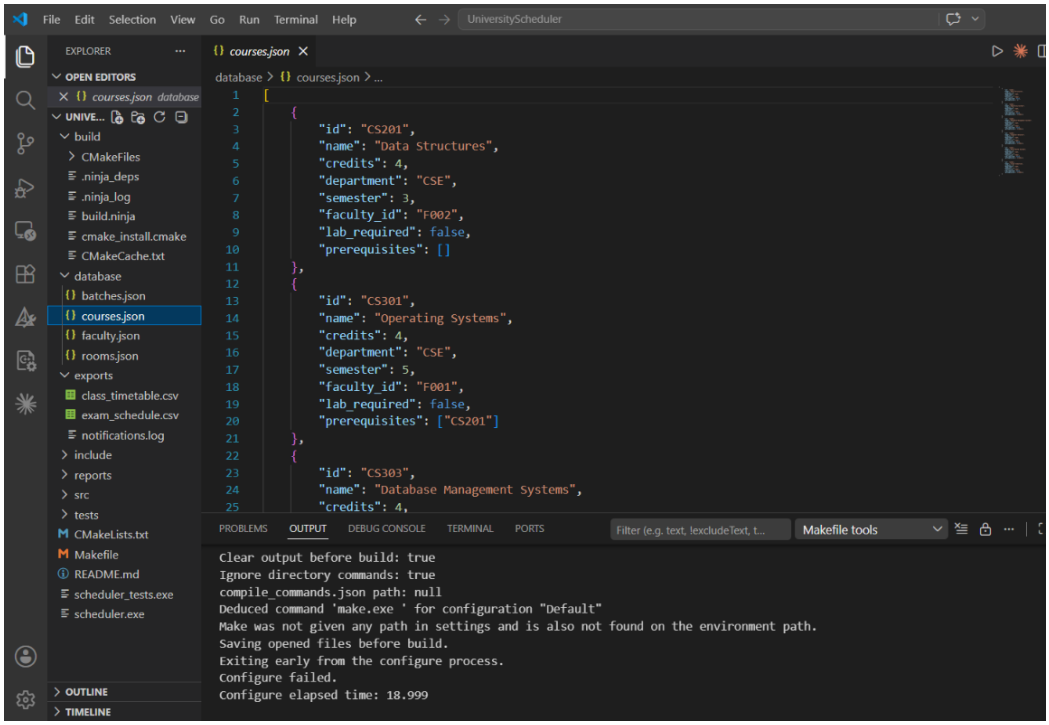


Fig 4.2: courses.json — course records with prerequisites and faculty assignment

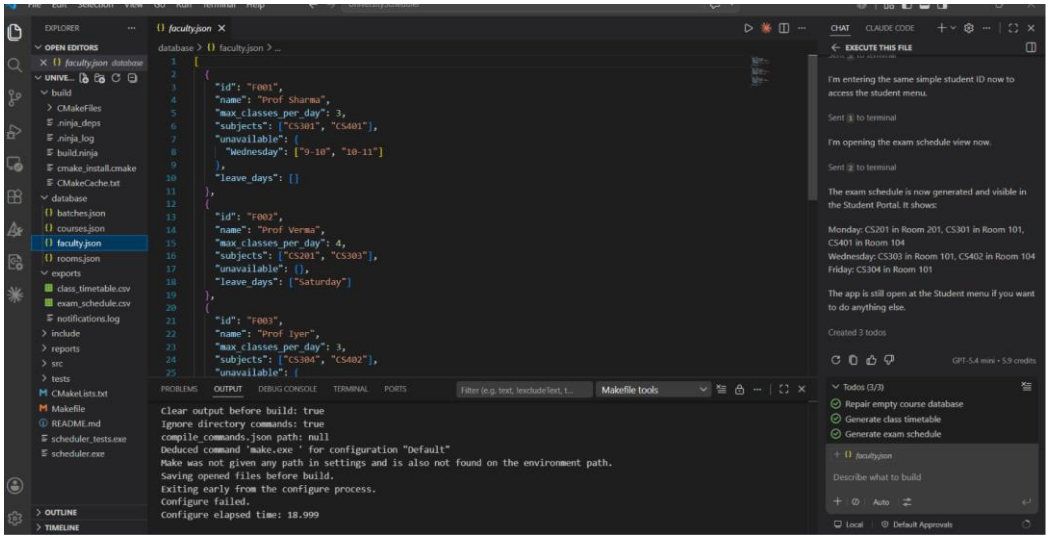


Fig 4.3: faculty.json — faculty availability and leave data

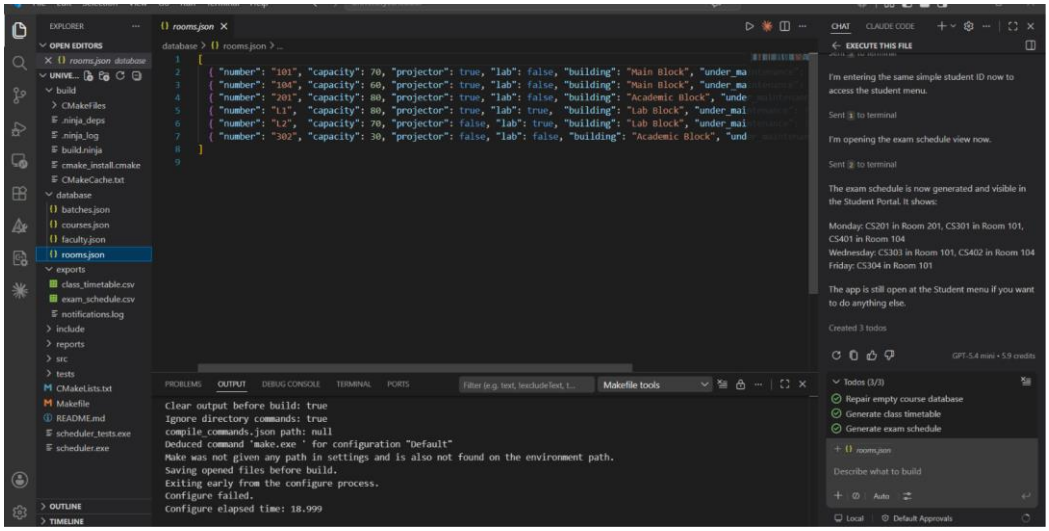


Fig 4.4: rooms.json — room capacity, lab, and maintenance flags

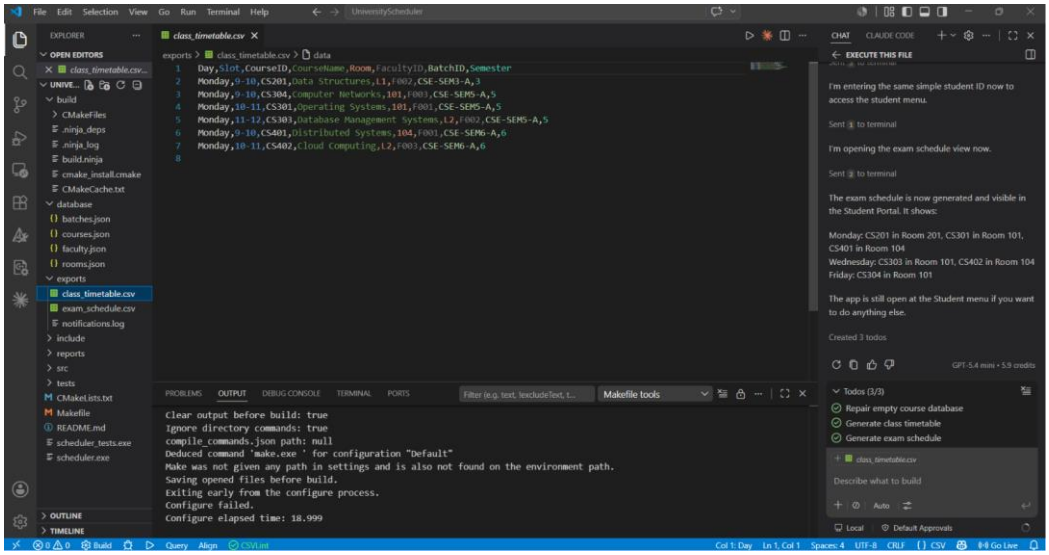


Fig 4.5: exports/class_timetable.csv — generated class timetable output

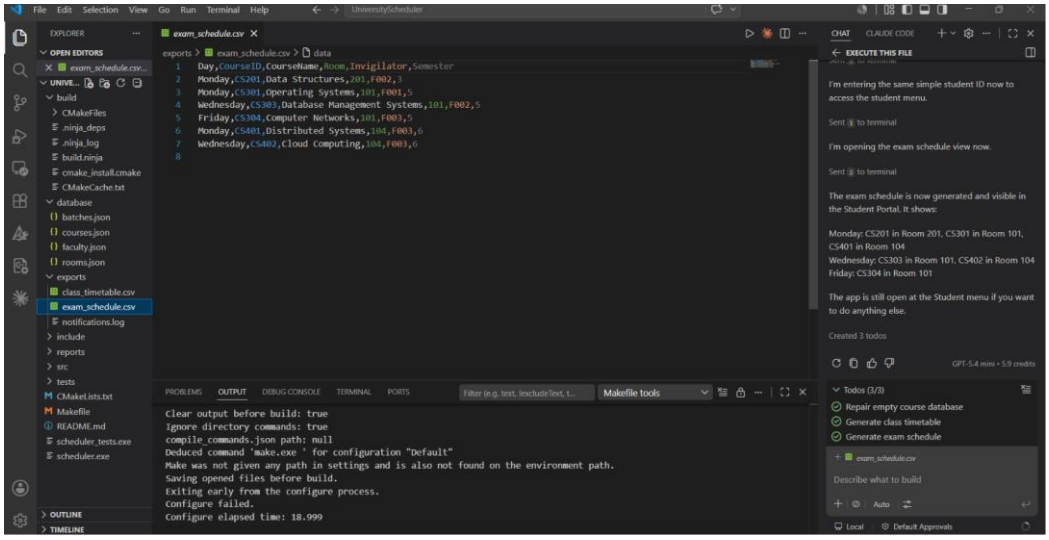


Fig 4.6: exports/exam_schedule.csv — generated exam schedule output

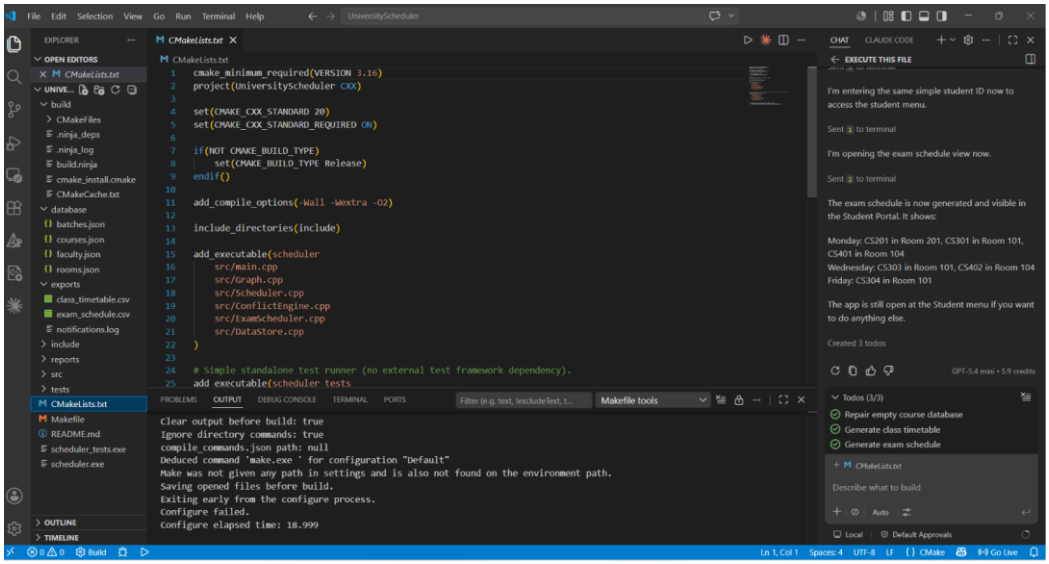


Fig 4.7: CMakeLists.txt — build configuration for the scheduler and test targets

BIBLIOGRAPHY

- nlohmann/json — JSON for Modern C++, https://github.com/nlohmann/json
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (Kahn's Algorithm for Topological Sorting).
- Project README and source documentation (this project), 2026.